NIRANTAR

GENOVATE 2.0 — AI Innovative Project Competition 2026

PRODUCT REQUIREMENTS DOCUMENT v2.0 — REAL-TIME IMPLEMENTATION SPECIFICATION

Goal: turn Nirantar from a polished railway intelligence prototype into an implementation-ready system with real railway telemetry, GPS, AI/ML, voice, maps, payments, caching and explicit failure handling.

1. Product Definition

Nirantar is a railway/passenger intelligence web platform combining live train telemetry, passenger GPS, station intelligence, predictive analytics, voice assistance and a sandboxed railway-service transaction flow.

• **LIVE:** fresh third-party railway telemetry.

• **CACHED:** last successful telemetry with its timestamp.

• **SIMULATED:** deterministic demo movement; never mislabeled as live.

• Railway data is a third-party integration; Nirantar must not claim that RailRadar is an official Indian Railways API.

2. Full Technology Stack

Layer	Technology	Role
Web	Next.js + React + TypeScript	App UI, routes, server API
UI	Tailwind CSS	Responsive premium interface
Map	Mapbox GL JS	Rail route, train and passenger layers
Rail data	RailRadar REST API	Live train/station telemetry
GPS	Browser Geolocation API	Passenger location
Backend	Next.js Route Handlers / Node	API proxy, secrets, normalization
Realtime	Server polling + SSE/short polling	Fan-out normalized telemetry
Validation	Zod	Validate external responses
AI/ML	Existing Nirantar models	ETA, delay, occupancy intelligence
LLM	Existing NVIDIA-hosted LLM API	Natural-language reasoning
STT	Deepgram	Voice input/transcription
TTS	Murf API	Spoken responses
Payments	Razorpay Test Mode	Sandbox railway transaction
Testing	Vitest/Jest + Playwright + k6/Locust	Unit, E2E, load/failure
Optional	Redis + PostgreSQL	Shared cache/history if needed
Deploy	Vercel or equivalent HTTPS Node host	Public demo

3. Real-Time Railway API — Primary Integration

3.1 RailRadar live train endpoint

Current RailRadar documentation specifies **GET https://api.railradar.in/v1/trains/{number}/live**. It documents live status, delay, current-location telemetry including segment progress/speed/bearing, next halt, platform information and timestamps. Authentication is a Bearer API key.

```
GET https://api.railradar.in/v1/trains/12919/live
Authorization: Bearer ${RAILRADAR_API_KEY}

Optional:
?date=YYYY-MM-DD
&authoritative;=true
&geometry;=true
&format;=geojson
&includeCoordinates;=true
```

The documented sandbox quota is 1,000 requests/month, with 401/404/429/503 failure states. Therefore the browser must **never poll the provider directly**; one server-side refresh should serve many viewers.

3.2 Supporting endpoints

• **Live station board:** GET /v1/stations/{code}/live

• **Route geometry:** GET /v1/trains/{number}/route?format=geojson&stops;=true

• **Station search:** GET /v1/lookup/search/stations?q=...

• **Trains between stations:** GET /v1/trains/between/{from}/{to}?live=true

3.3 Provider adapter

```
lib/railway/
railwayApiService.ts
normalize.ts
cache.ts
types.ts
providers/
railRadarProvider.ts
mockProvider.ts
```

```
interface RailwayDataProvider {
getLiveTrainStatus(trainNumber: string): Promise;
getTrainRoute(trainNumber: string): Promise;
getStationBoard(stationCode: string): Promise;
}
```

The UI consumes only normalized Nirantar objects. Provider-specific JSON never leaks into React components.

4. Normalized Live Telemetry

```
type DataMode = "LIVE" | "CACHED" | "SIMULATED";

interface NormalizedTelemetry {
trainNumber: string;
trainName?: string;
status: "running" | "at_station" | "scheduled" | "cancelled" | "unknown";
delayMinutes: number | null;
latitude: number | null;
longitude: number | null;
speedKmh: number | null;
bearingDegrees: number | null;
segmentProgress: number | null;
currentStationCode?: string;
nextStationCode?: string;
nextStationName?: string;
platform?: string | null;
providerUpdatedAt: string;
fetchedAt: string;
source: "railradar" | "mock";
mode: DataMode;
}
```

5. Exact Live Data Flow

```
Browser
↓ GET /api/trains/12919/live
Next.js server route
↓
railwayApiService
↓
RailRadar provider adapter
↓ HTTPS + Bearer key
RailRadar
↓ JSON
validate + normalize
↓
cache + telemetry engine
↓
NormalizedTelemetry
↓
Mapbox + ETA + alerts + AI + voice
```

• Server route calls RailRadar; client never sees the secret.

• Validate the response before use.

• Record both providerUpdatedAt and fetchedAt.

• Return LIVE on successful fresh fetch.

• On 429/503/timeout, return last good response as CACHED.

• If no cache exists, allow SIMULATED only when explicitly enabled.

6. Polling and Cache Policy

Competition recommendation: refresh the currently watched train every 15–30 seconds. Do not poll every second. If smooth animation is desired, interpolate visually between confirmed provider positions while retaining the last provider timestamp.

```
cache key:
railway:live:{trainNumber}:{journeyDate}

Refresh:
15–30 sec for active train
longer TTL for route geometry
one provider refresh → many browser viewers
```

Use in-memory cache for a single-instance demo. Use Redis if deployment is multi-instance. Do not spend the provider quota once per viewer.

7. Mapbox + Realtime Map

Mapbox GL JS should have three independent layers: route geometry, live train marker, and passenger GPS. The route is fetched/cached separately; telemetry only moves the train marker.

```
map.getSource("train-live").setData({
type: "FeatureCollection",
features: [{
type: "Feature",
geometry: { type: "Point", coordinates: [lng, lat] },
properties: { trainNumber, delayMinutes, speedKmh }
}]
});
```

8. Passenger GPS

GPS does not require a railway API. Use the browser Geolocation API after permission. Mapbox's GeolocateControl can track the user's location; production/demo hosting must use HTTPS.

```
navigator.geolocation.watchPosition(
position => {
const { latitude, longitude, accuracy, heading } = position.coords;
updatePassengerLocation({ latitude, longitude, accuracy, heading });
},
showGpsError,
{ enableHighAccuracy: true, maximumAge: 5000, timeout: 10000 }
);
```

Show GPS accuracy. Do not claim exact platform detection unless the accuracy and station geometry justify it.

9. AI/ML and Prediction

• Consume **NormalizedTelemetry**, not raw provider JSON.

• Use delay, speed, segment progress, route distance, next halt, schedule and station dwell as prediction signals.

• Baseline ETA can be remaining-distance/recent-speed; ML can correct the baseline.

• Every prediction should expose confidence or quality.

• LLM responses must be grounded in current telemetry; the LLM must not invent train positions.

10. Voice Architecture

```
Passenger speech
↓
Deepgram STT
↓
NVIDIA-hosted LLM / intent layer
↓
Nirantar controlled tools
■■ getLiveTrainStatus()
■■ getStationBoard()
■■ getPassengerContext()
↓
```

```
Answer text
↓
Murf streaming TTS
```

Deepgram handles speech input/transcription where enabled. Murf provides TTS/streaming speech; current Murf documentation recommends Falcon 2 for low-latency streaming. The voice layer should call Nirantar tools for live facts rather than allowing the LLM to hallucinate them.

11. Razorpay Test Transaction

Use **Razorpay Test Mode only**. Test mode has separate keys and does not process real payments. Make the transaction railway-specific: e.g. demo lounge/service pass, station service, or ticket add-on.

```
Select demo railway service
↓
Server creates Razorpay Test Order
↓
Razorpay Checkout
↓
Test success/failure
↓
Client confirmation + server verification
↓
Signed webhook → order/payment state
↓
Nirantar transaction status
```

Validate the webhook signature against the raw request body and deduplicate using Razorpay's event identifier. Never expose the secret key in the browser.

12. Environment Variables

```
RAILRADAR_API_KEY=...
MAPBOX_ACCESS_TOKEN=...
DEEPGRAM_API_KEY=...
MURF_API_KEY=...
NVIDIA_API_KEY=...
RAZORPAY_KEY_ID=rzp_test_...
RAZORPAY_KEY_SECRET=...
RAZORPAY_WEBHOOK_SECRET=...
```

Only server-side environment variables hold secrets. Never use NEXT_PUBLIC_* for secret credentials.

13. Internal Nirantar API

Endpoint	Method	Purpose
/api/trains/{number}/live	GET	Live/cached/simulated normalized telemetry
/api/trains/{number}/route	GET	Cached route geometry
/api/stations/{code}/live	GET	Live station board
/api/stations/search?q=	GET	Station autocomplete
/api/voice/transcribe	POST	Speech → text proxy
/api/voice/speak	POST	Text → TTS/stream
/api/payment/order	POST	Create test order
/api/payment/verify	POST	Verify checkout
/api/webhooks/razorpay	POST	Signed event handler

14. Failure Handling

• **401:** provider auth failure; show error, never fake LIVE.

• **404:** train/station not found.

• **429:** return cached data and show last update.

• **503/timeout:** return cached data; if unavailable, offer marked simulation.

• **GPS denied:** map remains usable without passenger marker.

• **LLM down:** deterministic railway lookup still works.

• **Murf down:** show text answer.

• **Razorpay down:** never fabricate a successful transaction.

15. Security

• HTTPS deployment.

• Server-side API keys only.

• Zod/runtime validation for external responses.

• Rate-limit Nirantar endpoints.

• Provider calls centralized through one service.

• Razorpay webhook signature validation + idempotency.

• Minimize retention of precise passenger GPS.

16. Minimal Data Model

Entity	Fields
TrainSnapshot	trainNumber, mode, status, delay, lat, lng, speed, bearing, providerUpdatedAt, fetchedAt
StationBoard	stationCode, trainNumber, arrival, departure, platform, delay, status
PassengerSession	sessionId, lat, lng, accuracy, lastSeenAt
Prediction	trainNumber, predictedEta, confidence, modelVersion, generatedAt
TransactionDemo	orderId, amount, status, paymentId, createdAt

A database is optional for the competition build. Start stateless + cache; add PostgreSQL only if history/accounts/analytics require persistence.

17. Testing

• **Unit:** provider JSON → NormalizedTelemetry.

• **Integration:** real train number → LIVE response or explicit provider error.

• **Cache:** force 429/503 → CACHED response.

• **Simulation:** deterministic movement and SIMULATED badge.

• **Map:** marker uses normalized coordinates; route is independent.

• **GPS:** permission granted/denied/error paths.

• **Voice:** speech → transcript → controlled data query → answer → TTS.

• **Payment:** test order + verification + signed webhook.

• **Load:** stress Nirantar's API, not the upstream provider. One refresh should fan out to many viewers.

18. Competition Demo Sequence

1. Open dashboard and show the operational map.

2. Select a real supported train number.

3. Show LIVE badge, delay, provider timestamp and route.

4. Refresh/observe marker movement driven by normalized telemetry.

5. Enable passenger GPS and show passenger marker separately.

6. Ask voice assistant: “Where is my train and how late is it?”

7. Show answer grounded in current telemetry.

8. Open live station board.

9. Trigger a controlled provider failure and demonstrate CACHED/SIMULATED state.

10. Run a Razorpay Test Mode railway-service transaction.

11. Explain the architecture from provider → normalized telemetry → map/AI/voice.

19. Implementation Priority

• **P0:** RailRadar key, provider adapter, normalization, /api/trains/{number}/live.

• **P0:** LIVE/CACHED/SIMULATED state machine.

• **P0:** Mapbox route + live marker.

• **P0:** server cache + upstream request deduplication.

• **P1:** passenger GPS + nearest-station logic.

• **P1:** live station board.

• **P1:** ETA/delay intelligence.

• **P1:** Deepgram → NVIDIA → controlled tools → Murf.

• **P2:** Razorpay Test Mode.

• **P2:** persistence/history/advanced analytics.

20. Definition of Done

• A real train-number request reaches the third-party railway provider through Nirantar's server.

• Response is validated and normalized.

• UI proves LIVE vs CACHED vs SIMULATED.

• Train route and live marker render on Mapbox.

• Passenger GPS works independently.

• AI/voice answers from controlled live data.

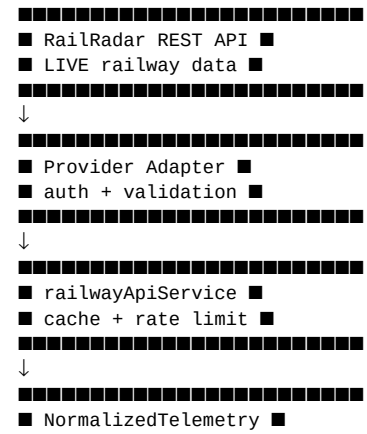
• Payment is sandboxed and uses no real money.

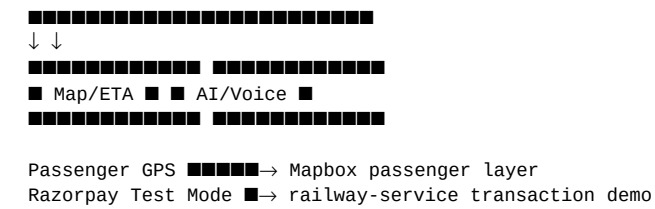
• Provider failure does not produce misleading LIVE data.

• No secret API keys appear in the client bundle.

• The complete demo is repeatable from a documented train number and test transaction.

21. Architecture at a Glance





Passenger GPS ■■■■■→ Mapbox passenger layer
Razorpay Test Mode ■→ railway-service transaction demo

22. External Reference Set

RailRadar live train status documentation specifies the live endpoint, Bearer authentication, telemetry fields, geometry options and 401/404/429/503 failure states. Its route endpoint supplies GeoJSON geometry and station stops, while its live station endpoint supplies arrival/departure status and platform information.

Mapbox documentation supports browser geolocation and repeated GeoJSON-source updates for realtime map data. Razorpay documents Test/Live mode separation, API keys and signed webhooks. Murf documents streaming TTS and Falcon 2 for low-latency voice. Deepgram documents realtime speech APIs.

Core rule: Nirantar's UI does not own the railway data source. The normalized backend telemetry layer does. This is what makes the system testable, cacheable, provider-independent and credible during judging.